

# Failure (a crashing tool, and a bad description) — Hints

---

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangChain/Python), **Advanced** (what a production tool library actually does differently). Read Basic first even if you already know this — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 1 — The idea, and the exact pieces

---

### Basic Version

This exercise has two separate halves — do them one at a time, don't mix them together.

**Half 1 — the crashing tool:** write a tool that raises an exception on purpose (like dividing by zero, or raising `ValueError` directly). First, call it and let it crash your whole program — see that happen once. Then wrap the body in `try/except` and return a plain error string instead of letting it raise, and see the model actually receive that string as the tool's result.

**Half 2 — the bad description:** write a tool with a genuinely vague, unhelpful description (just "gets data" or similar). Run an ambiguous prompt against it 5 times alongside a second tool, note the split, then rewrite the description to be precise, and run the same prompt 5 more times. Compare the two splits.

Things to use:

- A tool whose body can raise `( 1 / 0 )`, or a manual `raise ValueError("boom")`.
- `try/except Exception as e: return f"Error: {e}"` inside the tool.
- Two versions of the same tool's docstring: a vague one, then a precise one.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

**Half 1:** first, watch it actually crash — call your failing tool function directly, unwrapped, and let the exception happen. You want to have actually seen the raw traceback once before you hide it, so "catching the error" doesn't feel like skipping a step. Then the key line is `return f"Error: {e}"` inside an `except` block, instead of letting the exception propagate. This is a deliberate design choice — the tool's "successful" return type and its "error" return type should look the same to the code path that reads the result (both are just strings the model can react to), because the model has no other way to see what happened. After catching the error and returning a string, pass that string back into the conversation as the tool's result, and ask the model to respond — check that it reacts sensibly to the error text instead of treating it as a real answer.

**Half 2:** reuse the two ambiguous tools from the Intermediate ambiguity exercise (or write two new ones) — give one of them a genuinely bad description, run your existing 5-run trial function against it, then edit only the description (not the function body), and run the exact same trial again. Print each run's pick both before and after, and count the split as a ratio (like "4/5 picked tool A") rather than eyeballing it. Write out, for Half 2, exactly what you're going to hold constant between the "before" and "after" runs — the comparison only means something if everything else stays fixed except the description text.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Advanced Version

**Half 1's harder question: should `except Exception` really catch everything the same way?**

`except ZeroDivisionError` catches the failure you deliberately planted. A bare `except Exception as e: return f"Error: {e}"` also catches a genuine bug in *your own* tool code — a typo'd variable name, a `NameError`, an `AttributeError` from code you wrote wrong — and quietly turns it into a plausible-looking error string the model treats exactly like an expected service failure. That's a real production problem: an expected failure ("the API timed out") and an unexpected one ("my tool has a bug") look identical to whoever reads the result, and only one of those two things should ever happen again the same way.

The fix is to catch the *specific* exceptions you actually expect, separately from everything else — and for the "everything else" case, return a generic, safe message instead of `str(e)` (which might leak an internal file path, a variable name, or other implementation detail you didn't mean to show), while still logging the real exception somewhere you can see it.

**Half 2's harder question: is a 5-run split from one trial something you should actually act on?**

The Intermediate ambiguity exercise already raised this — it's worth applying here too, on the "before vs. after" comparison specifically. A 5-run "before: 4/5, after: 1/5" swing is a strong, believable signal. A "before: 3/5, after: 2/5" swing might just be noise. Before writing "the fix worked" anywhere real, ask whether the swing is big enough, at this sample size, to actually mean something.

The extra pieces needed for Half 1's fix:

- Two separate `except` blocks: one for the specific, expected exception type, one generic `except Exception` fallback with a safe, non-leaking message.
- `import logging` and `logging.exception(...)` (or your Doc01 `get_logger()`) inside the generic fallback, so the real error is still visible to *you*, even though the model only ever sees the safe version.

Sketch the two-`except`-block version yourself before checking Hint 2.

**Difference between Basic, Intermediate, and Advanced:** Basic and Intermediate get both halves working — a caught error instead of a crash, and a measured before/after split from a description fix. Advanced questions whether "catch everything the same way" and "trust one 5-run comparison" are actually good enough for real code — separating expected failures (caught specifically, reported honestly) from unexpected bugs (caught generically, reported safely, logged loudly), and being honest about how much a small sample can actually tell you.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 2 — The plan, and almost the whole thing

### Basic Version

Half 1:

```
define a tool that raises an error when called
call it directly, unwrapped -> watch it crash

wrap the body in try/except
on error, return "Error: " + the error message

register it on a model, ask something that triggers it
feed the tool's error string back to the model
print how the model responds to the error
```

Half 2:

```
write a tool with a vague description
run a 5-run selection trial -> record the split

change only the description to be precise
run the same 5-run trial again -> record the new split

print both splits side by side
```

Here's the tricky part for Half 1 — turning a crash into a clean returned string:

```
# tool_selection_practice.py – Failure section
from langchain_core.tools import tool

@tool
def flaky_divide(a: int, b: int) -> str:
    """Divide a by b."""
    try:
        result = a / b
        return str(result)
    except Exception as e:
        return f"Error: {e}"
```

For Half 2, the trickiest part is making the "before" and "after" comparison fair: use the exact same trial function, the exact same prompt, and change nothing but the docstring. Finish both halves yourself, then check the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

One small piece to get you unstuck for Half 2 — a description-swappable tool factory, so "change nothing but the docstring" is actually enforced by the code, not just by discipline:

```
# tool_selection_practice.py – Failure section
def make_tool_with_description(description: str):
    def get_weather(city: str) -> str:
        return f"Sunny in {city}"
    get_weather.__doc__ = description
    return tool(get_weather)
```

Use this to build the "before" and "after" versions of the same tool from one shared function body. Try finishing both halves yourself before looking at the full [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Advanced Version

Fill in the missing generic-fallback branch yourself — this is the piece that keeps an unexpected bug from looking exactly like an expected failure:

```
# tool_selection_practice.py – Failure section
import logging

from langchain_core.tools import tool

logger = logging.getLogger(__name__)

@tool
def flaky_divide(a: int, b: int) -> str:
    """Divide a by b. Returns an error message string if b is zero."""
    try:
        result = a / b
        return str(result)
    except ZeroDivisionError as e:
        return f"Error: {e}"
    except Exception:
        # your turn: log the real exception with logger.exception(...) so it's
        # not lost, but return a generic, safe string to the model – don't
        # include str(e) here, it might leak something internal
        ...
```

For Half 2's sample-size question, sketch a version that runs the before/after trial at a larger `runs` count (like 20 instead of 5) and reports the split as a percentage, before comparing all 3 of your finished versions against the [Solution](#).

**Difference between Basic, Intermediate, and Advanced:** same two halves (catch instead of crash, measure a description fix) at 3 completeness levels — Basic and Intermediate use one broad `except Exception` and trust a single 5-run comparison, while Advanced separates expected failures from unexpected bugs (with different, more honest handling for each) and checks whether the before/after swing is actually big enough to mean something.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)